第6章 第5节 Function Calling进阶实战

- 来源: https://class.imooc.com/lesson/5277#mid=104929
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第6周 Function Calling：工具调用机制
- 模块: Function Calling进阶实战
- 课节: 图文 1-1 Function Calling进阶实战

1-1 Function Calling进阶实战

第6章 第5节 Function Calling进阶实战

> [!TIP] 阅读指南 前两节学了 Function Calling 的基本原理和工作流程，但在实际项目中会碰到更多麻烦：LLM 选错工具、传错参数，多工具配合不当，还要兼顾成本、性能与健壮性。本节围绕这些实战问题，梳理一套可落地的工程方案。所有概念和伪代码都对应一份完整示例源码（samples/chapter6/robust_function_calling.py），可以一边看文档一边对照代码。

5.1 Function Calling的工程化防线

这一部分从工程视角梳理上一节“理想版 Function Calling”中尚未处理好的地方：包括调用流程中的错误与保护机制、工具与参数设计的坑、多工具协作时的职责边界，以及性能与成本上的隐性风险。要让 FC 真正跑在生产环境，还有很多细节要处理。

工具调用可能出错的场景

在实际使用中，Function Calling 会遇到各种错误：

```
场景1：LLM选错工具
用户："帮我算一下2的100次方"
LLM错误地调用：get_weather() ← 完全错误的工具

场景2：参数错误
用户："查一下北京天气"
LLM调用：get_weather(city=123) ← 参数类型错误

场景3：工具执行失败
用户："查询XXX角色信息"
工具返回：{"error": "角色不存在"} ← API返回错误

场景4：网络异常
调用外部API时超时或连接失败
```

如果不处理这些错误，整个流程就会崩溃。

基础错误处理

从工程视角看，要让 Function Calling 不“崩”，至少要在调用循环里补上几道保护：

- 使用 max_iterations 限制整体循环次数，防止 LLM 在工具调用和回答之间来回跳转造成死循环。
- 在每一轮中，对工具名称、参数解析、函数执行分别加上 try/except，把错误转化为结构化的信息返回给 LLM，而不是让程序直接抛异常退出。
- 在最外层再包一层 API 调用保护，一旦底层模型调用失败，就给出清晰的系统错误提示，而不是静默挂掉。

> [!TIP] 代码出处 以下伪代码节选自配套源码 robust_function_calling.py 中的 robust_function_call 函数，仅保留了错误处理相关的核心逻辑。

```
max_iterations = 5
iteration = 0

while iteration < max_iterations:
    # 1. 调用 LLM，拿到本轮的决策
    message = call_llm_with_tools(messages)

    # 2. 如果没有 tool_calls，说明可以直接给出最终回答
    if not message.tool_calls:
        return message.content

    # 3. 先把 LLM 的决策写入对话历史
    messages.append(message)

    # 4. 依次执行本轮的每个工具调用
    for tool_call in message.tool_calls:
        try:
            function_name = tool_call.function.name
            check_tool_exists(function_name)          # 验证工具存在

            arguments = parse_arguments(tool_call)    # 解析参数，可能失败
            result = run_function(function_name, arguments)  # 执行函数

            append_tool_result(messages, tool_call.id, result)  # 正常结果写回
        except Exception as e:
            append_tool_error(messages, tool_call.id, e)       # 出错信息写回，让 LLM 自行
处理

# 超过最大轮数仍未完成，给出兜底提示
return "处理超时，请简化问题后重试。"
```

上面代码中有一个关键设计：当工具执行失败时，错误信息以 {"error": "..."} 的格式返回给 LLM。

下面通过一个具体例子来看：

角色不存在的例子

```
用户："查一下雷电将军和八重神子哪个输出高？"

第1次LLM调用 → tool_calls = [
    get_character_info("雷电将军"),
    get_character_info("八重神子")
]

执行结果：
- get_character_info("雷电将军") → {"error": "角色不存在"}
- get_character_info("八重神子") → {"error": "角色不存在"}

第2次LLM调用 → 看到两个错误
LLM思考："数据库里没有这两个角色，我不能瞎编"
返回 content = "抱歉，数据库中没有雷电将军和八重神子的信息。
              目前支持的角色有：胡桃、甘雨..."
```

LLM 不知道工具调用失败了，可能基于空数据胡乱回答。

反过来，当 LLM 能收到错误信息，情况完全不同——LLM 知道查询失败的原因，可以给出准确的错误提示。

5.2 定义工具时对照这张清单

第2节已经详细讲过“如何定义一个好的工具”，这里不再重复原理，只给出一份可以在实战中快速对照的检查清单——写 tools JSON 定义时放在旁边当自检表。

- ☐ name 是否清晰且符合函数命名规范?
- ☐ description 是否说明了工具的用途?
- ☐ description 是否说明了适用场景?
- ☐ 每个参数是否都有 description?
- ☐ description 中是否给出了示例值?
- ☐ 是否标明了 required 参数?
- ☐ 类型定义是否准确?
- ☐ 能否使用 enum 限制参数范围?
- ☐ 工具之间的职责是否清晰不重叠?

如果 LLM 经常选错工具或传错参数，检查以下几点：

- 描述太模糊？ → 补充具体的使用场景和参数示例

描述太模糊？

- 功能重叠？ → 明确区分不同工具的职责边界

功能重叠？

- 参数太复杂？ → 考虑拆分成多个简单工具

参数太复杂？

- 缺少约束？ → 使用 enum 和 required 限制输入

```
enum
```

```
required
```

缺少约束？

5.3 多工具如何协作

单一职责原则

每个工具应该只做一件事，做好一件事。

```
# ✗ 不好的设计：一个工具做太多事
def character_analysis(character_name, include_equipment=True,
                       include_damage=True, include_team=True):
    """又查信息，又推荐装备，又算伤害，还分析队伍"""
    # 功能太复杂，LLM 很难正确使用
    pass

# ✓ 好的设计：拆分成多个专注的工具
def get_character_info(character_name):
    """只负责获取角色基础信息"""
    pass

def calculate_damage(character_name, weapon, artifact_set):
    """只负责计算伤害"""
    pass

def recommend_team(character_name):
    """只负责推荐队伍搭配"""
    pass
```

每个工具只做一件事，LLM 更容易理解每个工具的作用、参数更简单出错率更低、而且可以灵活组合而非强制执行所有功能。

工具要保持独立性

每个工具应该是独立可用的，不要在 description 中写明必须先调用其他工具。

```
# ✗ 不好的定义：硬编码依赖关系
{
    "name": "calculate_damage",
    "description": "计算伤害。必须先调用 get_character_info 获取角色数据。",
    # ← 问题：假设了固定的调用顺序，限制了灵活性
    "parameters": {...}
}

# ✓ 好的定义：工具独立可用
{
    "name": "calculate_damage",
    "description": "根据角色、武器和圣遗物计算伤害期望值。如果不指定装备，将使用当前装备。",
    # ← 只描述自己的功能，不提其他工具
    "parameters": {
        "character_name": "...",
        "weapon": "...（可选）",  # ← 通过默认参数实现灵活性
        "artifact_set": "...（可选）"
    }
}
```

可以这样理解为什么工具要保持独立：硬编码"先调用 A 再调用 B"的依赖关系，等于限制了组合灵活性，也会让 LLM 困惑——它完全能推断出"先查信息再算伤害"的顺序，不需要人工指定。

工具描述只说明自己能做什么即可。

自己能做什么

避免工具功能重叠

如果两个工具功能相似，LLM 会困惑，不知道该选哪个。

```
# ✗ 不好的设计：功能重叠
{
    "name": "get_weather",
    "description": "获取天气信息",
    ...
},
{
    "name": "query_weather",  # ← 和上面有什么区别？
    "description": "查询天气数据",
    ...
}
```

5.4 性能与成本控制

减少不必要的工具定义

只暴露用户真正需要的工具。

如果你的系统有20个内部函数，但用户只会用到其中5个，就只定义这5个工具。

```
# 假设你有这些函数
def get_character_info(...): pass
def get_weapon_info(...): pass
def get_artifact_info(...): pass
def calculate_damage(...): pass
def recommend_team(...): pass
def analyze_abyss(...): pass
def check_event(...): pass
# ... 还有更多

# ✗ 不好：把所有函数都暴露给 LLM
tools = [定义所有20个工具]
# LLM 需要从20个工具中选择，容易选错

# ✓ 好：只暴露高频核心工具
tools = [
    get_character_info,
    calculate_damage,
    recommend_team
]
# LLM 从3个工具中选择，准确率高
```

5.5 常见的坑与对策

LLM 可能会"创造"参数值

即使写了很详细的 description，LLM 有时还是会传入意料之外的值：

```
# 期望的调用
get_character_info("胡桃")

# LLM 可能传入
get_character_info("Hu Tao")          # 英文名
get_character_info("胡桃(火)")        # 带属性
get_character_info("香菱的好朋友")    # 描述性文字
```

在函数内部做映射和容错

详细实现请参考源码的 get_character_info 函数，核心逻辑：

```
# 名称标准化映射表
name_mapping = {
    "hu tao": "胡桃",
    "hutao": "胡桃",
    "胡桃(火)": "胡桃",
    "香菱的好朋友": "胡桃",
    # ...
}

# 统一转换为小写查找
normalized_name = character_name.lower().strip()
if normalized_name in name_mapping:
    character_name = name_mapping[normalized_name]
```

处理用户未说明参数

很多时候用户不会把所有参数说全，比如只说"查一下天气"，却没说城市和日期。这类情况本质上是"关键参数缺省"，需要在工具定义和实现里约定好如何补全或反问。

设置默认值

```
def get_weather(city: str = "上海", date: str = "今天"):
    """默认查询上海今天的天气"""
    ...
```

让 LLM 反问用户

在工具定义中说明：

```
{
    "description": "获取指定城市的天气。如果用户没有说明城市，请先询问用户所在城市。"
}
```

控制成本

分场景加载工具

```
if "天气" in user_input:
    tools = weather_tools
elif "角色" in user_input:
    tools = game_tools
else:
    tools = common_tools
```

缓存常见查询结果

```
import functools

@functools.lru_cache(maxsize=100)
def get_character_info(character_name: str):
    """同一角色的信息会被缓存"""
    return CHARACTER_DB.get(character_name)
```

lru_cache 装饰器详解

lru_cache 是 Python 内置的缓存装饰器，会自动记录函数的调用结果。

```
# 第1次调用：执行函数，查询数据库
result1 = get_character_info("胡桃")  # ← 真正执行

# 第2次调用同一参数：直接返回缓存结果
result2 = get_character_info("胡桃")  # ← 从缓存读取，极快！

# 不同参数：重新执行
result3 = get_character_info("甘雨")  # ← 真正执行
```

参数说明：maxsize=100 表示最多缓存 100 个不同的调用结果，超过后删除最久未使用的缓存（LRU = Least Recently Used）。

只适用于不变数据（如角色基础信息）；如果数据会变化（如实时天气），不要使用缓存。

5.6 一份完整的示例代码

前面几节从错误处理、工具定义、多工具协作、成本控制等维度拆解了"理想版 Function Calling"在工程上需要补齐的细节。这一节，把这些技巧全部落到一份可以直接运行的示例里，方便完整体验一个"健壮版" FC 流程：

> [!TIP] 本节配套完整示例源码：samples/chapter6/robust_function_calling.py

建议按下面的方式使用这份源码：

- 先跑通源码，体验一轮完整的工具调用对话。
- 对照前面各小节，分别在源码里找到对应的实现位置（错误处理、检查清单、多工具协作、缓存等）。
- 结合自己项目的需求，按“检查清单”的方式，一项项把这些防护和优化搬到真实系统中。

5.7 下一节预告

在这一节，“理想版 Function Calling” 升级成了可以跑在真实系统里的"健壮版"：补哪些防线、如何在代码里实现这些防线、如何用完整示例源码做自检，这些都已梳理清楚。但当系统越来越复杂、工具越来越多，仅靠手工维护 Function Calling 逻辑就会变得吃力——需要一种更"工程化"的方式来组织这些工具和后端能力。

现在，是时候把视角从"单个应用里的 Function Calling"提升到"整个系统的能力编排"了：如何在多服务、多工具的场景下，让 AI 安全地、可观测地调用一切？ 下一章将从这个问题出发，系统讲解 MCP（Model Context Protocol），看看它是如何在 Function Calling 之上，提供一套标准化的"工具接入与管理"方案的。

5.8 ■ 学点英语

中文	English	音标	说明
单一职责原则	Single Responsibility Principle	/ˈsɪŋgl rɪˌspɒnsə ˈbɪləti ˈprɪnsəpl/	每个工具只做一件事、做好一件事的工具设计原则
名称标准化映射	Name Normalization Mapping	/neɪm ˌnɔːməlaɪ ˈzeɪʃn ˈmæpɪŋ/	在函数内部处理LLM传入的变体名称，统一转换为标准值的容错策略
分场景加载	Scenario-based Tool Loading	/sɪˈnɑːriəʊ beɪst tuːl ˈləʊdɪŋ/	根据用户输入关键词预判业务场景、只加载该场景相关工具的性能优化策略
最近最少使用缓存	lru_cache	/lruː kæʃ/	Python内置缓存装饰器，自动记录函数调用结果，对不变数据节省重复查询成本
工程化防线	Engineering Guardrails	/ˌendʒɪˈnɪərɪŋ ˈgɑːdreɪlz/	围绕FC调用流程构建的错误处理、安全检查、超时兜底等工程保护机制